

State of Agentic Coding

Daniela Petruzalek

Developer Relations Engineer

Google Cloud

2026



Who am I?

Brazilian 🇧🇷, based in London 🇬🇧

Specialisation: backend (Python & Go),
data engineering and agentic AI

Hobbies: anime, games e cosplay

Blog: danicat.dev

LinkedIn: linkedin.com/in/petruzalek

Twitter: x.com/danicat83



Agenda



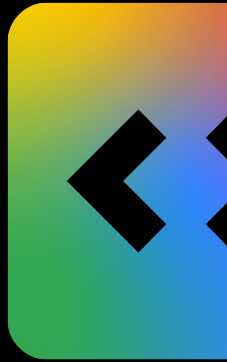
01 The 7 Stages of Agentic Coding

02 Agentic Surfaces

03 Tools and Techniques



The 7 stages of agentic coding



Using AI in the browser

Zero integration

- ✓ You treat AI as a separate entity
- ✓ Ask AI questions in a browser tab and **manually copy-paste** the occasional snippet back into your editor
- ✓ Low trust, low integration



Stage 2

Coding agent with manual approval

Check every single step

- ✓ You use Antigravity, Cursor, Windsurf, Copilot, or Claude for coding
- ✓ You manually approve every single tool call and carefully review **every file edit** and **shell command**



The guarded collaborator

Selective trust

- ✓ You begin to loosen the reins
- ✓ You allow **file edits** to happen automatically but **still manually approve** other tool calls
- ✓ You start building confidence in the agentic loop



Stage 4

The hands-off director

Focus on end results

- ✓ Trust goes up
- ✓ You stop reading every line of code as it's generated
- ✓ You switch to "Full Screen" mode
- ✓ You are no longer the writer; **you are the reviewer**



YOLO mode

Trust the agent

- ✓ You disable all restrictions and stop reviewing file edits and tool calls
- ✓ Your brain shifts from implementation details to high-level architectural guidance.



Stage 6

The orchestrator

Coordinate multiple agents

- ✓ You scale horizontally
- ✓ You run 3 to 5 agent instances in parallel
- ✓ Your focus shifts from "how to code" to "how to manage a fleet of agents"

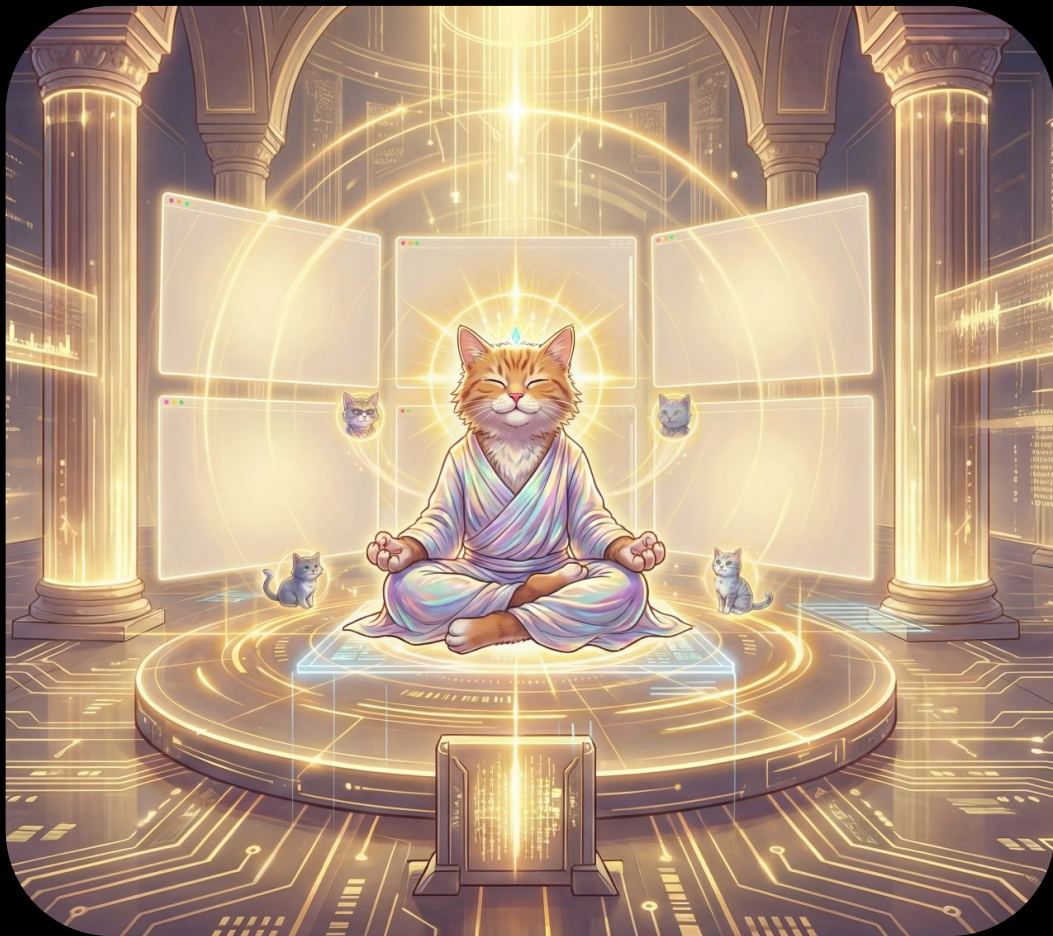


Stage 7

The architect

A new way of working

- ✓ You are no longer writing software; **you are guiding a system** that writes software
- ✓ You define the what and the why, and the **swarm** handles the how entirely in the background. Code is just an implementation detail



How do we increase trust?



Agentic Surfaces

Agentic Surfaces

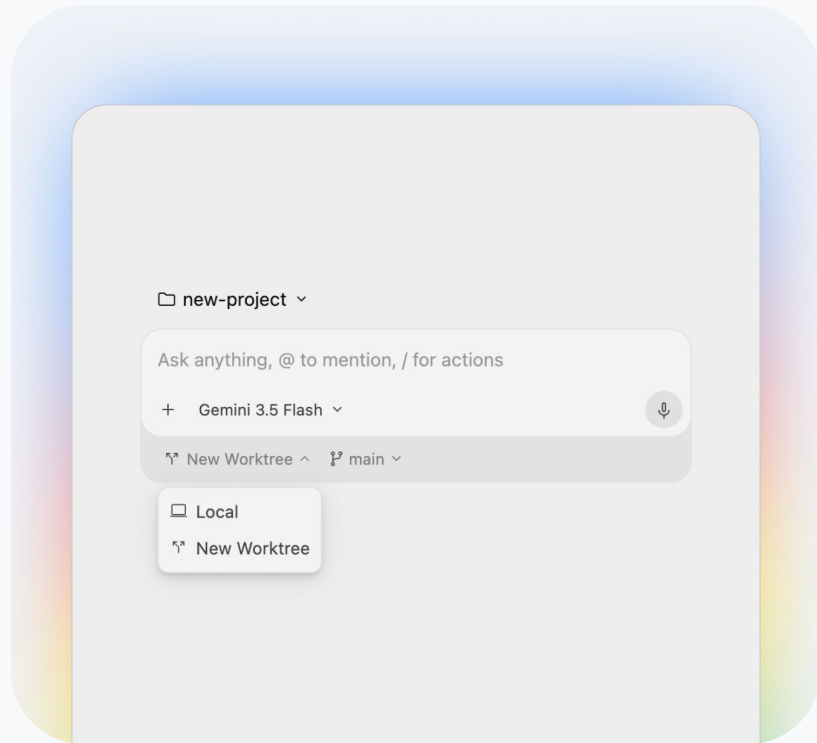
Code First: the **code** is in first plane; agent interaction via auto-complete and/or a side panel

Agent First: the **agent** is in first plane and code moves to the background

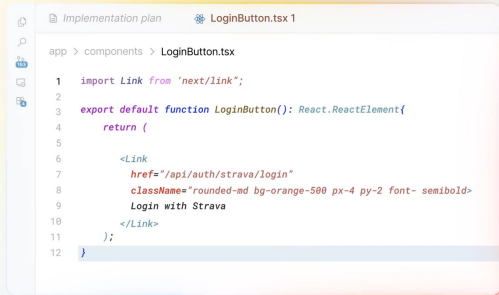
Headless Agents: agent runs in a cloud somewhere, always on

Antigravity 2.0

Command center to
manage multiple agents in
parallel.



Antigravity 2.0



Implementation plan LoginButton.tsx 1

app > components > LoginButton.tsx

```
1 import Link from 'next/link';
2
3 export default function LoginButton(): React.ReactElement{
4   return (
5
6     <Link
7       href="/api/auth/strava/login"
8       className="rounded-md bg-orange-500 px-4 py-2 font- semibold>
9       Login with Strava
10     </Link>
11   );
12 }
```

Antigravity IDE



Welcome to Antigravity CLI!

Choose your color scheme:

- light
- solarized light
- colorblind-friendly light
- > dark
- solarized dark
- colorblind-friendly dark
- tokyo night
- terminal

[Next]

> you: add a greeting function

AGY: Here's the change:

```
3 import "fmt"
4
5 - func main() {
5 + func greet(name string) {
6 +   fmt.Printf("Hello, %s!\n", name)
7 }
```

Antigravity CLI

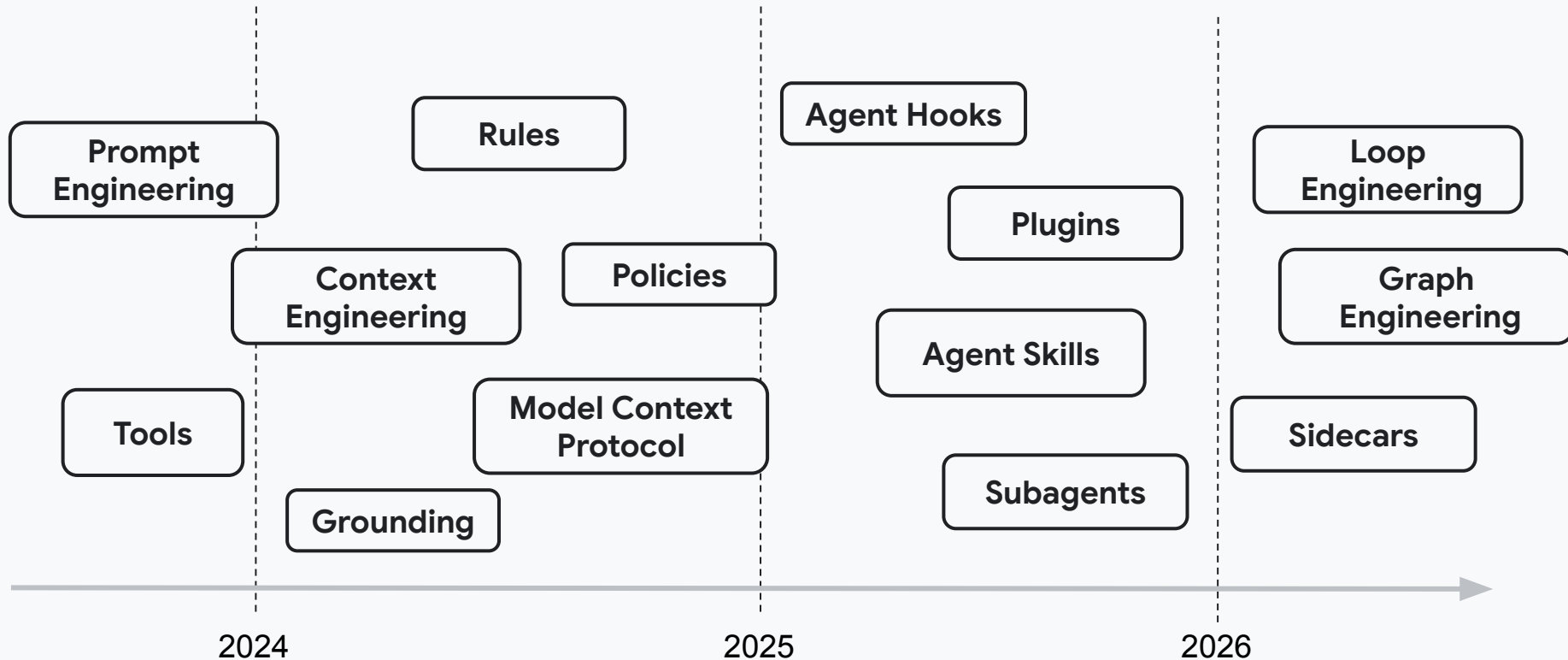


Antigravity SDK

What should be
in our Toolbox?



The Evolution of Coding Agents



Prompts

Ideal: context, task, constraints,
acceptance criteria

In practice:

freestyle + **/grill-me** + **/goal** + iteration

Plan mode: when you have high
technical certainty



Rules

Rules: AGENTS.md / GEMINI.md / CLAUDE.md

- Non-deterministic: bad place for **enforcing** behaviour
- Rules **rot** based on model and harness capabilities

Nowadays: keep it very lean

- Skills to activate (and keep active)
- Very specific local instructions

Keep **deterministic** processes as code / scripts

Hooks

Add deterministic steps to specific triggers

When: before or after

What: tool, model call, turn

Problems: agents are **very good** at **bypassing** them

Git hooks and CI/CD are better places for guardrails

Model Context Protocol (MCP)

MCP spec defines: tools, prompts and resources

- Most people only use tools
- Skills over MCP ([SEP-2640](#))

Too many MCPs is bad for context

Great for documentation retrieval

- Google Developer Knowledge
- Agent Skills MCP

For everything else: Skills + CLI

modelcontextprotocol.io

Agent Skills

Promise: progressive disclosure

Reality: use **slash commands** to force activation

/skill-name became an "unofficial" notation

Too many skills is **also** bad for context

Skills **not** really fast

Challenges: provenance; keeping them up to date

agentskills.io

Just-in-Time Skills



github.com/danicat/kungfu

What skills should we write?

LEVEL 1

Documentation

Foundational knowledge, guides, specifications, and written reference material.

LEVEL 2

Process

Workflows, standard operating procedures, execution steps, and deterministic scripts.

LEVEL 3

Traits

Agent behaviour, ways of working and persona attributes.

skills.danicat.dev

Subagents

Two main uses: **context isolation** and **parallelisation**

Dynamic agents beat static ones:

Static: "develop X. create a tech lead, a frontend developer and a backend developer"

Dynamic: "develop X. you have a budget of 20 agents, use a divide and conquer strategy"

TL;DR: Teach your **coordinator** "how to be a tech lead"

Sidecars

Processes that launch in parallel to the agent

Antigravity manages their lifecycle

Good for creating **custom extensions** to the development UI (with **/generative_ui**)

Limitation: only available on Antigravity 2.0

Agy Paint Studio

Sidecar: nodejs web server running on port 7788

Skill as a shortcut: [/paint-studio](#)

Antigravity uses [/generative_ui](#) to render the artifact

Session is identified by conversation ID

Server sends messages to Antigravity via [agentapi](#) CLI

github.com/danicat/agy-paint-studio

Plugins

New standard: agent-plugins.org

Bundle of **MCPs** and **Agent Skills**

Allows custom extensions:

- *com.example.client/...*

Note: in Antigravity sidecars are also supported:

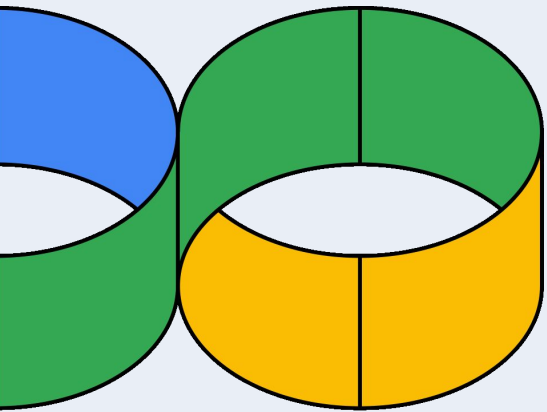
- *my-plugin/sidecars/my-sidecar*

```
my-plugin/  
├── plugin.json  
├── skills/  
│   ├── summarize/  
│   │   ├── SKILL.md  
│   │   ├── scripts/  
│   │   └── references/  
├── mcp.json  
└── com.example.client/  
    └── hooks/
```

Takeaways

Rot is the keyword of the moment: keep your prompts, rules and skills sharp

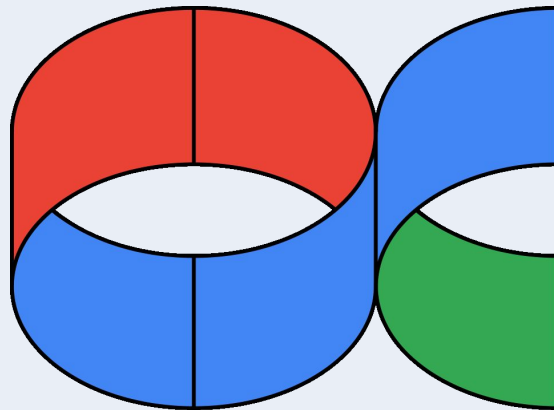
Personalised software is the ~~future~~ **present**. Build tools to improve your own development flow



Google Developer Program

Build the future

builders.google



Thank you

Blog: danicat.dev

LinkedIn: linkedin.com/in/petruzalek

Twitter: x.com/danicat83

Github: github.com/danicat/devoxx2026

